FOOD DELIVERY APP

```
import java.util.*;
// FoodItem Class
class FoodItem {
    private int id;
    private String name;
    private double price;
    public FoodItem(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    public void setId(int id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setPrice(double price) { this.price = price; }
    @Override
    public String toString() {
        return "FoodItem ID: " + id + " Name: " + name + " Price: " + price;
    }
}
// User Base Class
class User {
    private int userId;
    private String username;
    private long contactNo;
    public User(int userId, String username, long contactNo) {
        this.userId = userId;
```

```
        this.username = username;
        this.contactNo = contactNo;
    }
    public int getUserId() { return userId; }
    public String getUsername() { return username; }
    public long getContactNo() { return contactNo; }
    @Override
    public String toString() {
        return "User ID: " + userId + " Name: " + username;
    }
}
// Cart Class using Map
class Cart {
    private Map<FoodItem, Integer> items;
    public Cart() {
        items = new HashMap<>();
    }
    public void addItem(FoodItem foodItem, int quantity) {
        if(items.containsKey(foodItem))
            items.put(foodItem, items.get(foodItem) + quantity);
        else
            items.put(foodItem, quantity);
    }
    public void removeItem(FoodItem foodItem) {
        items.remove(foodItem);
    }
    public Map<FoodItem, Integer> getItems() {
        return items;
    }
    @Override
    public String toString() {
```

```
        String result = "\nCart Items:\n";
        for(Map.Entry<FoodItem,Integer> entry : items.entrySet()) {
            result += entry.getKey().getName() +
                    " Quantity: " + entry.getValue() + "\n";
        }
        return result;
    }
}
// Customer Class (Inheritance)
class Customer extends User {

    private Cart cart;
    public Customer(int userId, String username, long contactNo) {
        super(userId, username, contactNo);
        cart = new Cart();
    }
    public Cart getCart() {
        return cart;
    }
}
// DeliveryPerson Class
class DeliveryPerson {
    private int deliveryPersonId;
    private String name;
    private long contactNo;
    public DeliveryPerson(int id, String name, long contactNo) {
        this.deliveryPersonId = id;
        this.name = name;
        this.contactNo = contactNo;
    }
```

```
    public int getId() { return deliveryPersonId; }
    public String getName() { return name; }
    public long getContactNo() { return contactNo; }
    @Override
    public String toString() {
        return "DeliveryPerson ID: " + deliveryPersonId +
                " Name: " + name;
    }
}
// Restaurant Class using List
class Restaurant {
    private int id;
    private String name;
    private List<FoodItem> menu;
    public Restaurant(int id, String name) {
        this.id = id;
        this.name = name;
        menu = new ArrayList<>();
    }
    public void addFoodItem(FoodItem item) {
        menu.add(item);
    }
    public void removeFoodItem(int id) {
        menu.removeIf(item -> item.getId() == id);
    }
    public List<FoodItem> getMenu() {
        return menu;
    }
    @Override
    public String toString() {
```

```
        String result = "\nRestaurant: " + name + "\nMenu:\n";
        for(FoodItem item : menu) {
            result += item + "\n";
        }
        return result;
    }
}
// Order Class
class Order {
    private int orderId;
    private Customer customer;
    private Map<FoodItem,Integer> items;
    private String status;
    private DeliveryPerson deliveryPerson;
    private String deliveryAddress;

    public Order(int orderId, Customer customer) {
        this.orderId = orderId;
        this.customer = customer;
        items = new HashMap<>();
        status = "Pending";
    }
    public void addItem(FoodItem item, int quantity) {
        items.put(item, quantity);
    }
    public void setStatus(String status) {
        this.status = status;
    }
    public void setDeliveryPerson(DeliveryPerson dp) {
        this.deliveryPerson = dp;
    }
```

```
    public void setDeliveryAddress(String address) {
        this.deliveryAddress = address;
    }
    @Override
    public String toString() {
        String result = "\nOrder ID: " + orderId;
        result += "\nCustomer: " + customer.getUsername();
        result += "\nStatus: " + status;
        result += "\nAddress: " + deliveryAddress;
        if(deliveryPerson != null)
            result += "\nDelivery Person: " + deliveryPerson.getName();
        result += "\nItems:\n";
        for(Map.Entry<FoodItem,Integer> entry : items.entrySet()) {
            result += entry.getKey().getName() +
                    " Quantity: " + entry.getValue() + "\n";
        }
        return result;
    }
}
// Main Class
public class FoodDeliverySystem {
    public static void main(String[] args) {
        // Create Food Items
        FoodItem pizza = new FoodItem(1,"Pizza",250);
        FoodItem burger = new FoodItem(2,"Burger",120);
        FoodItem coke = new FoodItem(3,"Coke",50);
        // Create Restaurant
        Restaurant res = new Restaurant(101,"Food Zone");
        res.addFoodItem(pizza);
        res.addFoodItem(burger);
```

```
        res.addFoodItem(coke);
        System.out.println(res);
        // Create Customer
        Customer customer =
            new Customer(1,"Shree",9876543210L);
        // Add to cart
        customer.getCart().addItem(pizza,2);
        customer.getCart().addItem(coke,1);

        System.out.println(customer.getCart());

        // Create Delivery Person
        DeliveryPerson dp =
            new DeliveryPerson(201,"Ravi",9999999999L);

        // Create Order
        Order order = new Order(5001,customer);

        order.addItem(pizza,2);
        order.addItem(coke,1);

        order.setDeliveryAddress("Bangalore");
        order.setDeliveryPerson(dp);
        order.setStatus("Out for Delivery");

        // Print Order
        System.out.println(order);
    }
}
```